

The issue of your Next.js fallback proxy returning a custom 404 page with a 200 OK status usually occurs because Next.js handles the custom 404 page as a successful render within its own application context, or the rewrite configuration is incorrect and the request is not reaching the intended destination.  Medium +1

Troubleshooting Steps

1. **Verify Rewrite Configuration:** Ensure your `next.config.js` file is in the project root and configured correctly. Make sure your rewrite rules are ordered properly, with static assets and more specific rules coming before any general fallback or catch-all rules.

javascript



```
// next.config.js
module.exports = {
  async rewrites() {
    return [
      // Add specific rules first if needed
      // ...

      // Fallback proxy rewrite
      {
        source: '/*',
        destination: `https://your-destination-url.com/*`, // Use an absolute URL
        fallback: true, // You may not need 'fallback' here for simple proxy
      },
    ];
  },
};
```

2. **Restart Development Server:** Configuration changes in `next.config.js` require a server restart to take effect, not just hot reload.
3. **Check the Destination URL:** Verify that the `destination` URL in your rewrite configuration is correct and publicly accessible. You can test the destination URL directly in your browser or with a tool like `curl` to confirm it returns the expected content and a 200 OK status.
4. **Use an Absolute URL:** When proxying to an external service, use an absolute URL (e.g., `https://your-destination-url.com/*`) in the `destination` field to ensure the request is properly forwarded and not treated as a local path within your Next.js application.
5. **Examine Server-Side Logic (if applicable):**
 1. If you are using the `app` router, the `notFound()` function should be called within a server component or an API route when data is not found, which will trigger the correct 404 status.

2. If you are using the `pages` router, returning `{ notFound: true }` from `getStaticProps` or `getServerSideProps` will correctly signal to Next.js to return a 404 status code.
6. **Avoid Root `loading.tsx` File:** In some Next.js versions (especially Next.js 13+), having a `loading.tsx` file in the root `app` directory has been linked to issues where a 404 page returns a 200 OK status. Try removing or scoping the loading file if this is the case.
7. **Check Network Tab:** Use your browser's developer tools to inspect the network request. Look for the request to your proxy path and verify its `Type` (it should typically be `document` for pages) and the final `Status Code` returned from the server.  Stack Overflow +4

By ensuring your rewrite is correctly configured and pointing to a valid destination, you should be able to prevent Next.js from falling back to its local 404 handler and instead successfully proxy the request.